Concepts

Cache

The `Cache` is a central in-memory database that stores and manages all trading-related data, from market data to order history to custom calculations.

The Cache serves multiple purposes:

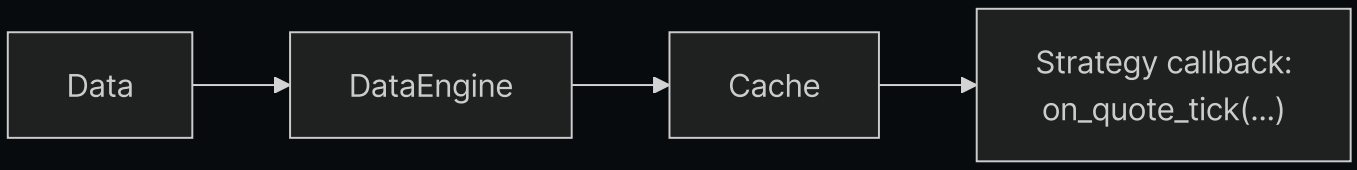
1. **Stores market data**:
   - Stores recent market history (e.g., order books, quotes, trades, bars).
   - Gives you access to both current and historical market data for your strategy.
2. **Tracks trading data**:
   - Maintains complete `Order` history and current execution state.
   - Tracks all `Position`s and `Account` information.
   - Stores `Instrument` definitions and `Currency` information.
3. **Stores custom data**:
   - You can store any user-defined objects or data in the `Cache` for later use.
   - Enables data sharing between different strategies.

How caching works

Built-in types:

- The system automatically adds data to the `Cache` as it flows through.
- In live contexts, the engine applies updates asynchronously, so you might see a brief delay between an event and its appearance in the `Cache`.

- For quotes, trades, and bars the `DataEngine` writes to the `Cache` before publishing to subscribers, so the latest value is available in the cache by the time your handler runs. Order book deltas and depth snapshots are published directly without a cache write; book state is maintained separately through `BookUpdater` subscriptions:



For the full step-by-step trace, see Data flow: life of a quote tick.

Basic example

Within a strategy, you can access the `Cache` through `self.cache`. Here's a typical example:

> Within a `Strategy` class, `self` refers to the strategy instance.

```
def on_bar(self, bar: Bar) -> None:
    # Current bar is provided in the parameter 'bar'

    # Get historical bars from Cache
    last_bar = self.cache.bar(self.bar_type, index=0)        # Last bar (practically the same as
    previous_bar = self.cache.bar(self.bar_type, index=1)    # Previous bar
    third_last_bar = self.cache.bar(self.bar_type, index=2)  # Third last bar

    # Get current position information
    if self.last_position_opened_id is not None:
        position = self.cache.position(self.last_position_opened_id)
        if position.is_open:
            # Check position details
            current_pnl = position.unrealized_pnl

    # Get all open orders for our instrument
    open_orders = self.cache.orders_open(instrument_id=self.instrument_id)
```

Configuration

Use the `CacheConfig` class to configure the `Cache` behavior and capacity. You can provide this configuration either to a `BacktestEngine` or a `TradingNode`, depending on your environment context.

Here's a basic example of configuring the `Cache`:

```
from nautilus_trader.config import CacheConfig, BacktestEngineConfig, TradingNodeConfig

# For backtesting
engine_config = BacktestEngineConfig(
    cache=CacheConfig(
        tick_capacity=10_000,  # Store last 10,000 ticks per instrument
        bar_capacity=5_000,    # Store last 5,000 bars per bar type
    ),
)

# For live trading
node_config = TradingNodeConfig(
    cache=CacheConfig(
        tick_capacity=10_000,
        bar_capacity=5_000,
    ),
)
```

By default, the `Cache` keeps the last 10,000 bars for each bar type and 10,000 trade ticks per instrument. These limits provide a good balance between memory usage and data availability. Increase them if your strategy needs more historical data.

Configuration options

The `CacheConfig` class supports these parameters:

```
from nautilus_trader.config import CacheConfig

cache_config = CacheConfig(
    database: DatabaseConfig | None = None,  # Database configuration for persistence
    encoding: str = "msgpack",               # Data encoding format ('msgpack' or 'json')
    timestamps_as_iso8601: bool = False,     # Store timestamps as ISO8601 strings
    buffer_interval_ms: int | None = None,   # Buffer interval for batch operations
    bulk_read_batch_size: int | None = None, # Batch size for bulk reads (e.g., MGET)
    use_trader_prefix: bool = True,          # Use trader prefix in keys
    use_instance_id: bool = False,           # Include instance ID in keys
    flush_on_start: bool = False,            # Clear database on startup
    drop_instruments_on_reset: bool = True,  # Clear instruments on reset
    tick_capacity: int = 10_000,             # Maximum ticks stored per instrument
    bar_capacity: int = 10_000,              # Maximum bars stored per each bar-type
)
```

Each bar type maintains its own separate capacity. For example, if you're using both 1-minute and 5-minute bars, each stores up to `bar_capacity` bars. When `bar_capacity` is reached, the `Cache` automatically removes the oldest data.

Database configuration

For persistence between system restarts, you can configure a database backend.

When is it useful to use persistence?

- **Long-running systems**: If you want your data to survive system restarts, upgrading, or unexpected failures, having a database configuration helps to pick up exactly where you left off.
- **Historical insights**: When you need to preserve past trading data for detailed post-analysis or audits.
- **Multi-node or distributed setups**: If multiple services or nodes need to access the same state, a persistent store helps ensure shared and consistent data.

```
from nautilus_trader.config import DatabaseConfig

config = CacheConfig(
    database=DatabaseConfig(
        type="redis",            # Database type
        host="localhost",        # Database host
        port=6379,               # Database port
        connection_timeout=2,    # Connection timeout (seconds)
        response_timeout=2,      # Response timeout (seconds)
    ),
)
```

Using the cache

Accessing market data

The `Cache` provides a full interface for accessing order books, quotes, trades, and bars. All market data in the cache uses reverse indexing, so the most recent entry sits at index 0.

Bar access

```
# Get a list of all cached bars for a bar type
bars = self.cache.bars(bar_type)  # Returns list[Bar] or an empty list if no bars found

# Get the most recent bar
latest_bar = self.cache.bar(bar_type)  # Returns Bar or None if no such object exists

# Get a specific historical bar by index (0 = most recent)
second_last_bar = self.cache.bar(bar_type, index=1)  # Returns Bar or None if no such object exis
```

```
# Check if bars exist and get count
bar_count = self.cache.bar_count(bar_type)  # Returns number of bars in cache for the specified b
has_bars = self.cache.has_bars(bar_type)    # Returns bool indicating if any bars exist for the s
```

Quote ticks

```
# Get quotes
quotes = self.cache.quote_ticks(instrument_id)                    # Returns list[QuoteTick] or a
latest_quote = self.cache.quote_tick(instrument_id)               # Returns QuoteTick or None if
second_last_quote = self.cache.quote_tick(instrument_id, index=1)  # Returns QuoteTick or None if

# Check quote availability
quote_count = self.cache.quote_tick_count(instrument_id)  # Returns the number of quotes in cache
has_quotes = self.cache.has_quote_ticks(instrument_id)    # Returns bool indicating if any quotes
```

Trade ticks

```
# Get trades
trades = self.cache.trade_ticks(instrument_id)         # Returns list[TradeTick] or an empty list
latest_trade = self.cache.trade_tick(instrument_id)    # Returns TradeTick or None if no such obj
second_last_trade = self.cache.trade_tick(instrument_id, index=1)  # Returns TradeTick or None if

# Check trade availability
trade_count = self.cache.trade_tick_count(instrument_id)  # Returns the number of trades in cache
has_trades = self.cache.has_trade_ticks(instrument_id)    # Returns bool indicating if any trades
```

Order book

```
# Get current order book
book = self.cache.order_book(instrument_id)  # Returns OrderBook or None if no such object exists

# Check if order book exists
has_book = self.cache.has_order_book(instrument_id)  # Returns bool indicating if an order book e

# Get count of order book updates
update_count = self.cache.book_update_count(instrument_id)  # Returns the number of updates recei
```

Price access

```
from nautilus_trader.core.rust.model import PriceType

# Get current price by type; Returns Price or None.
price = self.cache.price(
    instrument_id=instrument_id,
    price_type=PriceType.MID,  # Options: BID, ASK, MID, LAST
```

```
)
```

Bar types

```
from nautilus_trader.core.rust.model import PriceType, AggregationSource

# Get all available bar types for an instrument; Returns list[BarType].
bar_types = self.cache.bar_types(
    instrument_id=instrument_id,
    price_type=PriceType.LAST,  # Options: BID, ASK, MID, LAST
    aggregation_source=AggregationSource.EXTERNAL,
)
```

Simple example

```
class MarketDataStrategy(Strategy):
    def on_start(self):
        # Subscribe to 1-minute bars
        self.bar_type = BarType.from_str(f"{self.instrument_id}-1-MINUTE-LAST-EXTERNAL")  # examp
        self.subscribe_bars(self.bar_type)

    def on_bar(self, bar: Bar) -> None:
        bars = self.cache.bars(self.bar_type)[:3]
        if len(bars) < 3:   # Wait until we have at least 3 bars
            return

        # Access last 3 bars for analysis
        current_bar = bars[0]    # Most recent bar
        prev_bar = bars[1]       # Second to last bar
        prev_prev_bar = bars[2]  # Third to last bar

        # Get latest quote and trade
        latest_quote = self.cache.quote_tick(self.instrument_id)
        latest_trade = self.cache.trade_tick(self.instrument_id)

        if latest_quote is not None:
            current_spread = latest_quote.ask_price - latest_quote.bid_price
            self.log.info(f"Current spread: {current_spread}")
```

Trading objects

The `Cache` provides access to all trading objects within the system, including:

- Orders
- Positions
- Accounts

- Instruments

Orders

You can access and query orders through multiple methods, with flexible filtering options by venue, strategy, instrument, and order side.

Basic order access

```
# Get a specific order by its client order ID
order = self.cache.order(ClientOrderId("O-123"))

# Get all orders in the system
orders = self.cache.orders()

# Get orders filtered by specific criteria
orders_for_venue = self.cache.orders(venue=venue)                    # All orders for a specif
orders_for_strategy = self.cache.orders(strategy_id=strategy_id)     # All orders for a specif
orders_for_instrument = self.cache.orders(instrument_id=instrument_id)  # All orders for an instr
```

Order state queries

```
# Get orders by their current state
open_orders = self.cache.orders_open()                   # Orders currently active at the ven
closed_orders = self.cache.orders_closed()               # Orders that have completed their l
emulated_orders = self.cache.orders_emulated()           # Orders being simulated locally by
inflight_orders = self.cache.orders_inflight()           # Orders submitted (or modified) to
local_active_orders = self.cache.orders_active_local()   # Orders still managed locally (init

# Check specific order states
exists = self.cache.order_exists(client_order_id)           # Checks if an order with the given
is_open = self.cache.is_order_open(client_order_id)         # Checks if an order is currently op
is_closed = self.cache.is_order_closed(client_order_id)     # Checks if an order is closed
is_emulated = self.cache.is_order_emulated(client_order_id)  # Checks if an order is being simula
is_inflight = self.cache.is_order_inflight(client_order_id)  # Checks if an order is submitted or
is_active_local = self.cache.is_order_active_local(client_order_id)  # Checks if an order is stil
```

Order statistics

```
# Get counts of orders in different states
open_count = self.cache.orders_open_count()                  # Number of open orders
closed_count = self.cache.orders_closed_count()              # Number of closed orders
emulated_count = self.cache.orders_emulated_count()          # Number of emulated orders
inflight_count = self.cache.orders_inflight_count()          # Number of inflight orders
local_active_count = self.cache.orders_active_local_count()  # Number of locally active orders (i
total_count = self.cache.orders_total_count()                # Total number of orders in the syst

# Get filtered order counts
buy_orders_count = self.cache.orders_open_count(side=OrderSide.BUY)  # Number of currently open B
```

```
venue_orders_count = self.cache.orders_total_count(venue=venue)      # Total number of orders for
```

Positions

The `Cache` maintains a record of all positions and offers several ways to query them.

Position access

```
# Get a specific position by its ID
position = self.cache.position(PositionId("P-123"))

# Get positions by their state
all_positions = self.cache.positions()            # All positions in the system
open_positions = self.cache.positions_open()      # All currently open positions
closed_positions = self.cache.positions_closed()  # All closed positions

# Get positions filtered by various criteria
venue_positions = self.cache.positions(venue=venue)                     # Positions for a speci
instrument_positions = self.cache.positions(instrument_id=instrument_id)  # Positions for a speci
strategy_positions = self.cache.positions(strategy_id=strategy_id)        # Positions for a speci
long_positions = self.cache.positions(side=PositionSide.LONG)             # All long positions
```

Position state queries

```
# Check position states
exists = self.cache.position_exists(position_id)        # Checks if a position with the given ID
is_open = self.cache.is_position_open(position_id)      # Checks if a position is open
is_closed = self.cache.is_position_closed(position_id)  # Checks if a position is closed

# Get position and order relationships
orders = self.cache.orders_for_position(position_id)       # All orders related to a specific pos
position = self.cache.position_for_order(client_order_id)  # Find the position associated with a
```

Position statistics

```
# Get position counts in different states
open_count = self.cache.positions_open_count()      # Number of currently open positions
closed_count = self.cache.positions_closed_count()  # Number of closed positions
total_count = self.cache.positions_total_count()    # Total number of positions in the system

# Get filtered position counts
long_positions_count = self.cache.positions_open_count(side=PositionSide.LONG)             # Num
instrument_positions_count = self.cache.positions_total_count(instrument_id=instrument_id)  # Num
```

Accounts

```
# Access account information
account = self.cache.account(account_id)       # Retrieve account by ID
account = self.cache.account_for_venue(venue)  # Retrieve account for a specific venue
account_id = self.cache.account_id(venue)      # Retrieve account ID for a venue
```

Instruments and currencies

Instruments

```
# Get instrument information
instrument = self.cache.instrument(instrument_id) # Retrieve a specific instrument by its ID
all_instruments = self.cache.instruments()        # Retrieve all instruments in the cache

# Get filtered instruments
venue_instruments = self.cache.instruments(venue=venue)             # Instruments for a specific
instruments_by_underlying = self.cache.instruments(underlying="ES")  # Instruments by underlying

# Get instrument identifiers
instrument_ids = self.cache.instrument_ids()                  # Get all instrument IDs
venue_instrument_ids = self.cache.instrument_ids(venue=venue)  # Get instrument IDs for a specifi
```

Purging cached data

Long-running sessions accumulate closed orders, closed positions, account events, and unused instruments. The cache exposes targeted and bulk purge methods so strategies and the live trading engine can keep memory bounded without restarting the system.

Targeted purges

Use these to drop a single entity. Each refuses to purge while the entity is still active.

- `cache.purge_order(client_order_id)`: removes the order and every order-keyed index entry. Skips open orders.
- `cache.purge_position(position_id)`: removes the position, its snapshots, and position-keyed index entries. Skips open positions.
- `cache.purge_instrument(instrument_id)`: removes the instrument and every per-instrument map (order book, quotes, trades, mark/index/funding prices, instrument status, greeks, and bars referencing the instrument). Skips while any associated order is non-terminal (anything that has not reached a closed state, including initialized, submitted, accepted, emulated, released, and inflight orders) or any associated position is non-closed.

```
class HousekeepingStrategy(Strategy):
```

```
    def on_start(self) -> None:
        # Drop instruments that are no longer in the watchlist.
        for instrument_id in self.cache.instrument_ids(venue=self.venue):
            if instrument_id not in self.watchlist:
                self.cache.purge_instrument(instrument_id)
```

⚠ `purge_instrument` is intended for actors and strategies with their own lifecycle logic for deciding when an instrument is no longer needed. Purging an instrument that another component still relies on causes missing instrument lookups and loses market-data history. Active subscriptions belong to the data engine, so unsubscribe before purging if you no longer want updates.

Bulk purges

Use these to sweep older entries by age. They take the current timestamp and a buffer or lookback window in seconds.

- `cache.purge_closed_orders(ts_now, buffer_secs)`: closed orders whose close timestamp is older than `buffer_secs`.
- `cache.purge_closed_positions(ts_now, buffer_secs)`: closed positions whose close timestamp is older than `buffer_secs`.
- `cache.purge_account_events(ts_now, lookback_secs)`: account state events older than `lookback_secs`. A value of `0` purges all events.

Automatic purging in live trading

`LiveExecEngineConfig` schedules the bulk purges on a timer. Set the interval to enable the loop and the buffer or lookback to control how recent entries are protected. The following defaults work well for most live sessions:

```
from nautilus_trader.config import LiveExecEngineConfig

exec_engine = LiveExecEngineConfig(
    purge_closed_orders_interval_mins=15,
    purge_closed_orders_buffer_mins=60,
    purge_closed_positions_interval_mins=15,
    purge_closed_positions_buffer_mins=60,
    purge_account_events_interval_mins=15,
    purge_account_events_lookback_mins=60,
)
```

A 60-minute buffer keeps recent activity available for reconciliation while still trimming long-tail growth. Tune these down for HFT sessions and up if you need longer historical lookbacks

for analytics. See Configure live trading: memory management for the full parameter reference.

> The instrument purge has no automatic loop because the right time to drop an instrument depends on strategy state, not age. Call `cache.purge_instrument` from the actor or strategy that owns the instrument's lifecycle.

Custom data

The `Cache` can also store and retrieve custom data types in addition to built-in market data and trading objects. Use it to share any user-defined data between system components, primarily actors and strategies.

Basic storage and retrieval

```
# Call this code inside Strategy methods (`self` refers to Strategy)

# Store data
self.cache.add(key="my_key", value=b"some binary data")

# Retrieve data
stored_data = self.cache.get("my_key")  # Returns bytes or None
```

For more complex use cases, the `Cache` can store custom data objects that inherit from the `nautilus_trader.core.Data` base class.

> The `Cache` is not designed to be a full database replacement. For large datasets or complex querying needs, consider using a dedicated database system.

Best practices and common questions

Cache vs. portfolio usage

The `Cache` and `Portfolio` components serve different but complementary purposes in NautilusTrader:

Cache:

- Maintains the historical knowledge and current state of the trading system.
- Updates immediately when local state changes (for example, initializing an order before submission).
- Updates asynchronously as external events occur (for example, when an order fills).
- Provides a complete history of trading activity and market data.
- Keeps every event the strategy receives in the cache.

Portfolio:

- Aggregates position, exposure, and account information.
- Provides current state without history.

Example:

```
class MyStrategy(Strategy):
    def on_position_changed(self, event: PositionEvent) -> None:
        # Use Cache when you need historical perspective
        position_history = self.cache.position_snapshots(event.position_id)

        # Use Portfolio when you need current real-time state
        current_exposure = self.portfolio.net_exposure(event.instrument_id)
```

Cache vs. strategy variables

Choosing between storing data in the `Cache` versus strategy variables depends on your specific needs:

Cache storage:

- Use for data that needs to be shared between strategies.
- Best for data that needs to persist between system restarts.
- Acts as a central database accessible to all components.
- Ideal for state that needs to survive strategy resets.

Strategy variables:

- Use for strategy-specific calculations.
- Better for temporary values and intermediate results.

- Provides faster access and better encapsulation.
- Best for data that only your strategy needs.

Example:

The following example shows how you might store data in the `Cache` so multiple strategies can access the same information.

```
import pickle

class MyStrategy(Strategy):
    def on_start(self):
        # Prepare data you want to share with other strategies
        shared_data = {
            "last_reset": self.clock.timestamp_ns(),
            "trading_enabled": True,
            # Include any other fields that you want other strategies to read
        }

        # Store it in the cache with a descriptive key
        # This way, multiple strategies can call self.cache.get("shared_strategy_info")
        # to retrieve the same data
        self.cache.add("shared_strategy_info", pickle.dumps(shared_data))
```

Another strategy can retrieve the cached data as follows:

```
import pickle

class AnotherStrategy(Strategy):
    def on_start(self):
        # Load the shared data from the same key
        data_bytes = self.cache.get("shared_strategy_info")
        if data_bytes is not None:
            shared_data = pickle.loads(data_bytes)
            self.log.info(f"Shared data retrieved: {shared_data}")
```

Related guides

- Data - Data types stored in the cache.
- Strategies - Strategies access cache for market data and state.
- Reports - Generate reports from cached data.

‹ Positions
Previous Page

Message Bus ›
Next Page